Hierarchical Graph Drawing Algorithms

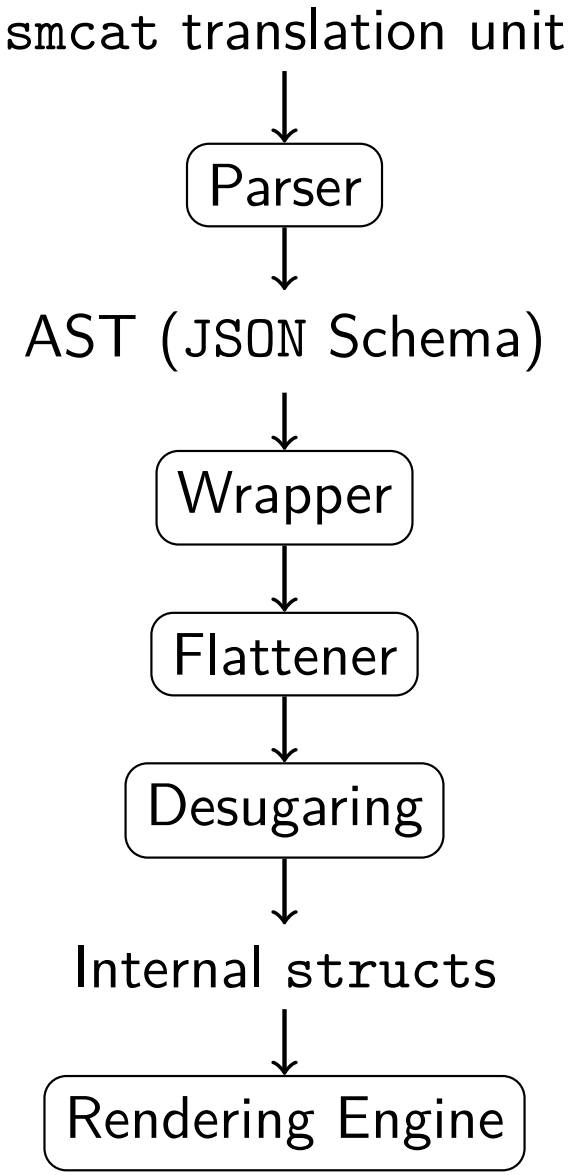
Anish Teja Bramhajosyula

International Institute of Information Technology Bangalore

Introduction

- **`smcat`** is a complete pipeline to read (supported) statechart modelling languages and render them in specified output formats.
- Input formats supported: `smcat`, `scxml`, `json`.
- Output formats supported: `ast`, `dot`, `eps`, `json`, `oldeps`, `oldps`, `oldps2`, `oldsvg`, `pdf`, `png`, `ps`, `ps2`, `scjson`, `scxml`, `smcat`, `svg`.

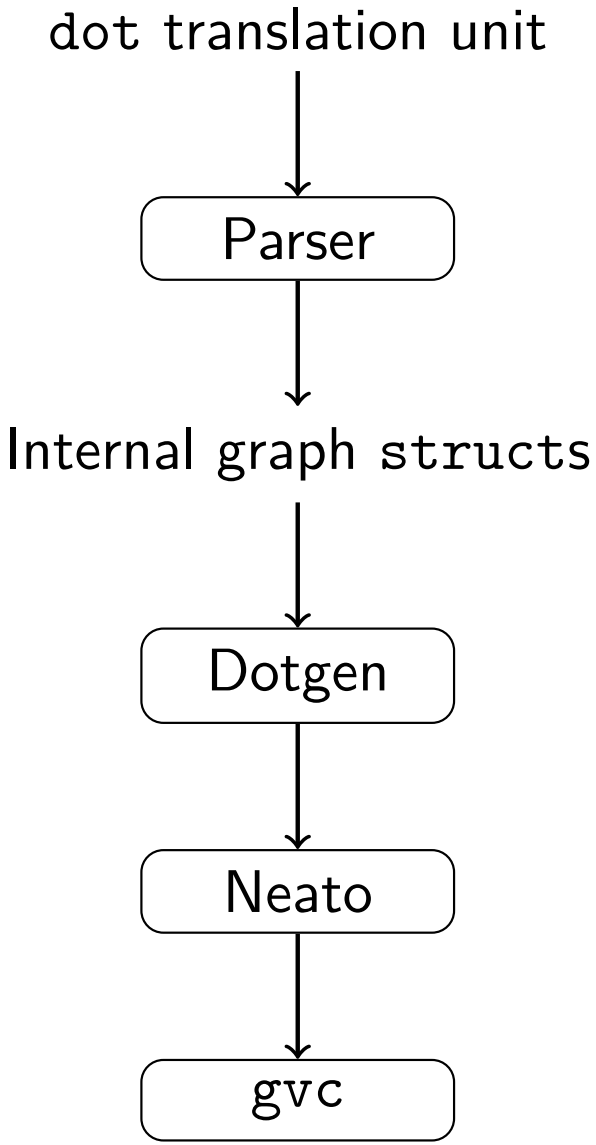
Execution Pipeline



Graphviz

- Graph Visualisation Software developed by AT&T
- Uses a modelling language called `dot`
- Employs various algorithms for layouting the vertices of a graph, assigning physical coördinates and drawing edges suitably

Execution Pipeline



Attaches mathematical drawing data

Performs post-processing and attaches physical coördinates

Writes image data

Dotgen

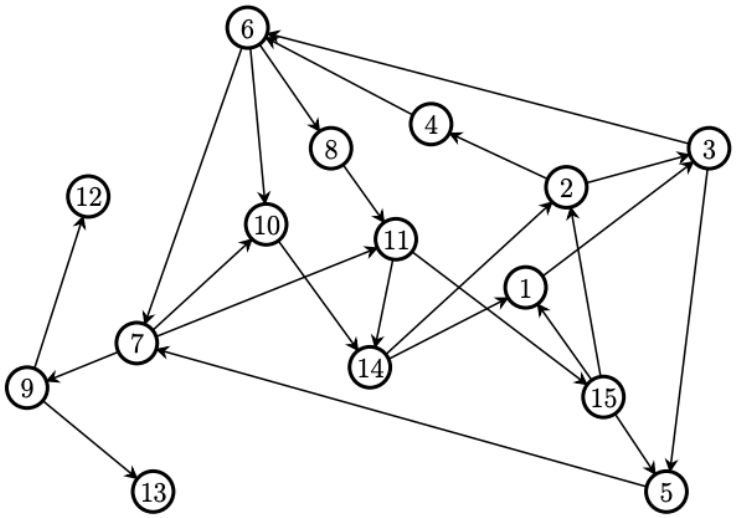
- Converts to directed acyclic graph (DAG)
- Utilises the Sugiyama framework:
  - Vertex Ranking/Levelling (assign vertices to levels)
  - Minimise number of edge crossings
  - Straighten the edges — route the edges and calculate coördinates
- Passes resulting data to `neato` to attach physical coördinates

Sugiyama Framework

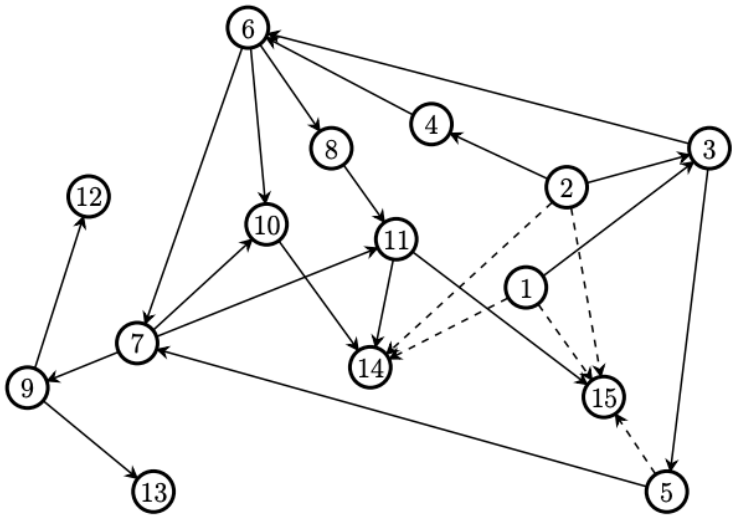
Goal: Given a directed acyclic graph (DAG):

1. Edges must point in a uniform direction.
2. Short & straight edges are more readable.
3. Uniformly distributed vertices avoid clutter.
4. Edge crossings obstruct comprehension.

Example graph drawn using the Sugiyama framework

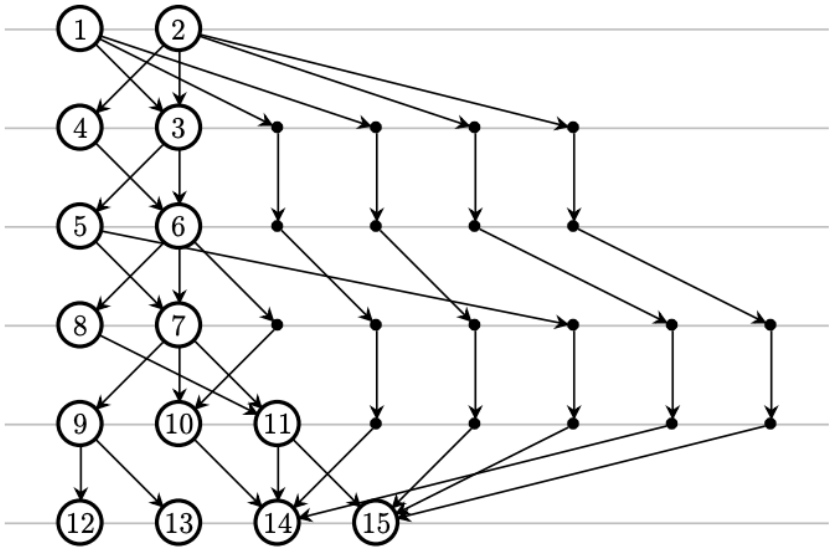


(a) Input digraph, G.

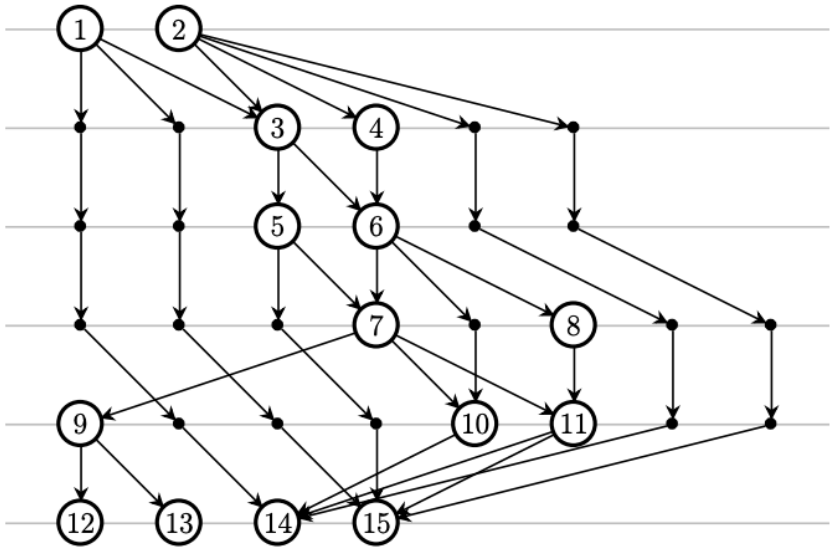


(b) Cycles removed.

Example graph drawn using the Sugiyama framework

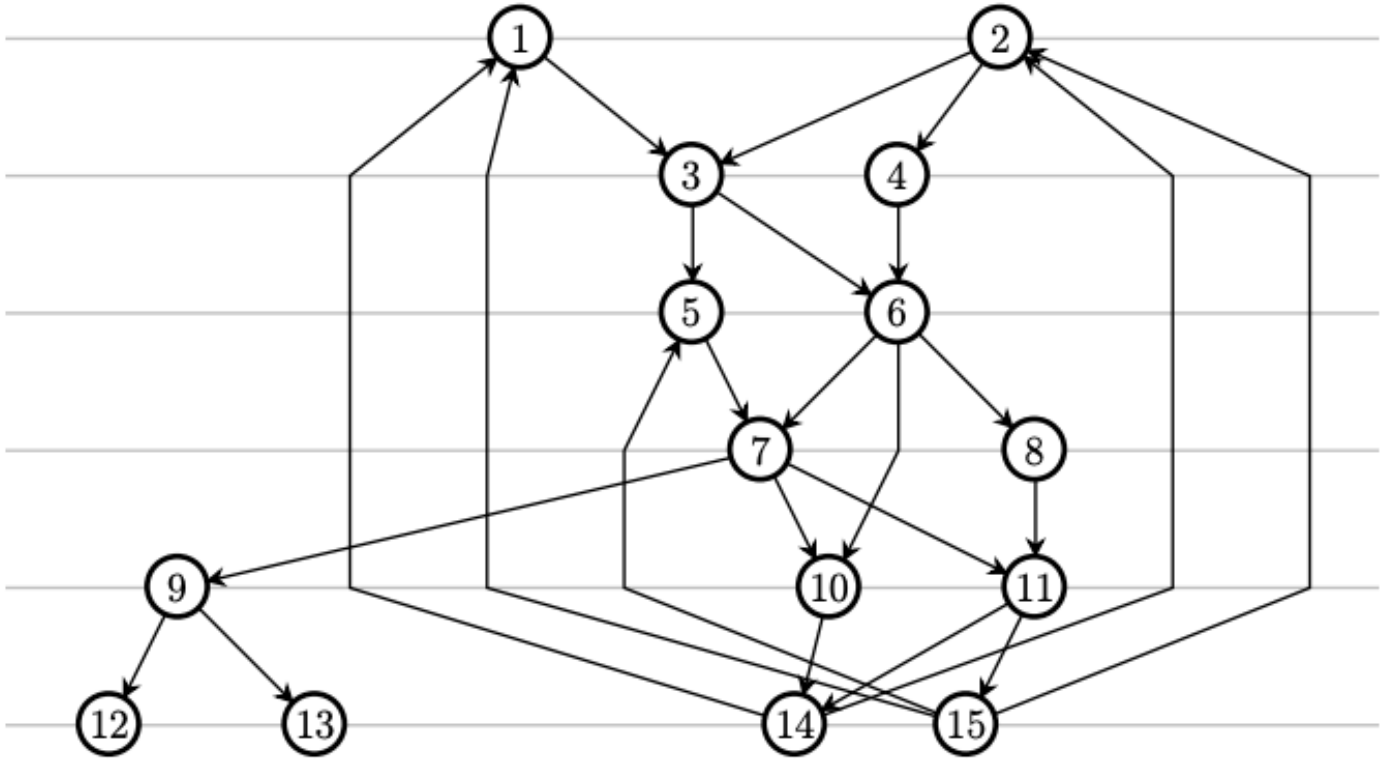


(c) After leveling.



(d) Edge crossings minimized.

Example graph drawn using the Sugiyama framework



(e) Edges straightened.

Cycle Removal — Preprocessing

Definition

A graph (V, E) has a ***two–cycle*** if $\exists u, v \in V : (u, v) \in E \wedge (v, u) \in E$

- It is assumed that the input digraph has no two–cycles.
- While the `dotgen` engine removes offending edges which make the digraph cyclic, it is an extension to the theoretical framework, and is feasible because the removed edges only affect its internal rendering and not the actual graph itself and can be added back later. In the theoretical model, offending edges are only reversed, since working on a subgraph of the original digraph (with missing edges) may have undesirable results.

Definition

A set of edges whose removal makes a digraph acylic is called a ***Feedback Arc Set*** **(FAS).**

Definition

A set of edges whose reversal makes a digraph acyclic is called a ***Feedback Set*** **(FS).**

Lemma

Given an edge set E, if $E_0 \subseteq E$ is an FS, then E_0 is also an FAS.

Proof.

We prove by contradiction. Let $G := (V, E)$ be the original graph, G_{FS} be the graph obtained by reversing the edges in E_0, and G_{FAS} be the graph obtained by removing the edges in E_0.
Suppose that E_0 is an FS, i.e, G_{FS} is a DAG, but E_0 is not an FAS, hence G_{FAS} is not a DAG, i.e., it contains at least one directed cycle, say C.
Let E_1 be the set of edges in E_0 with reversed directions. Then, the edge set of G_{FS} is $(E \setminus E_0) \cup E_1$.
$\because E \setminus E_0 \subseteq (E \setminus E_0) \cup E_1 \Rightarrow$ Since C is contained in G_{FAS}, all edges of C are in $E \setminus E_0$, hence C must be present in G_{FS} as well.
But, G_{FS} is a DAG, which is a contradiction.

$\square$

Lemma

It is always possible and easy to find an FS for a digraph.

Proof.

This proof can be generalised by induction.
Suppose $G := (V, E)$ and $V := \{v_1 \dots v_n\}$. Let $(i_1 \dots i_n)$ be any permutation of $(1 \dots n)$ such that we get an arbitrary ordering $(v_{i_1} \dots v_{i_n})$ of V. Now, any edge in E must either go forward or backward in this ordering, and cycles are not possible, as opposite directions are partitioned into two different subsets. This partitions E into two sets, say E_0 and E_1, respectively. If all the edges in one of the subsets is reversed, then all the edges in E point forward, and hence no cycle exists anymore in G. Thus, we have converted G to a DAG using a subset of edges (FS) in polynomial time.

$\square$

- A typical requirement for an FS is to have the minimum number of edges.
- Finding a minimum–cardinality FS, which is equivalent to finding a minimum-cardinality FS, is known to be NP–hard.
- We use heuristics to solve these problems.
- The following algorithms are based on vertex–ordering:
  - DFS Approach
  - Vertex Out–Degree Approach
  - Divide & Conquer Approach
  - Berger–Shor Algorithm
  - Greedy Cycle Removal
- A greedy algorithm with heuristics based on breaking cycles edge–by–edge is also covered.

Heuristics based on Vertex–Ordering

Depth–First Search Approach

Theorem (stated without proof)

A digraph has a cycle if and only if its DFS tree has a back edge.

- It can be shown that there exist vertex orderings of a digraph for which the FS partitions have the same size.
- Hence, such an ordering can be found by performing DFS in linear time.
- Based on the above theorem, the back edges in the DFS tree form the FS.
- The number of back edges is at most $|E| - |V| + 1$, which is expensive for dense graphs, since $|E| \to |V|^2$.

Heuristics based on Vertex–Ordering

Vertex Out–Degree Approach

- According to Eades *et al.*, 1993, the vertices are ordered from top to bottom, with non–increasing out–degree at every vertex. This ordering can be computed in linear time and is shown to give smaller FSs than the DFS method.
- The vertices are partitioned such that all the edges point in one direction in an ordering (in the DAG, using the output FS), so that they naturally *flow* from top to bottom. Hence, we place high out–degree vertices at the top and low at the bottom.

Divide & Conquer Approach

Eades_DAG

Input: Graph $G := (V, E)$
Output: FS V_1, V_2

Label vertices of G as $\{i, i+1, \ldots, i+|V|-1\}$

procedure $\textsc{Ordering}(V, E, i) :=$
    if $|V| = 1$ **then**
        $\textsc{label}(V, i)$
        return
    end if

    $k \leftarrow$ **if** $|V|$ is even **then** $\frac{|V|}{2}$ **else** 1
    $V_1, V_2 \leftarrow \textsc{partition}(V) : \forall v_1 \in V_1, v_2 \in V_2, \Delta_G^+(v_1) \geq \Delta_G^+(v_2)$

    call $\textsc{ordering}(V_1, E, i)$
    call $\textsc{ordering}(V_2, E, i + |V_1|)$
end procedure

Heuristics based on Vertex–Ordering

Divide & Conquer Approach

- Running Time: $O(\min\{(|V| + |E|) \lg |V|, |V|^2\})$
- Observed to produce 20% better results, especially for dense digraphs

Heuristics based on Vertex–Ordering

Berger–Shor Algorithm

```
Input: Digraph G := (V, E)
Output: Vertex–Ordering
S_l ← ∅
S_r ← ∅
while G do
    while ∃v ∈ V : Δ⁻_G(v) = 0 do //sink
        V ← V \ {v}
        prepend v to S_r
    end while

    while ∃v ∈ V : Δ⁺_G(v) = 0 do //source
        V ← V \ {v}
        append v to S_l
    end while

    if G ≠ ∅ then //cycles
        choose v : Δ⁺_G(v) − Δ⁻_G(v) is maximum
        V ← V \ {v}
        append v to S_l
    end if
end while
return S_1 ∪ S_2
```

Heuristics based on Vertex–Ordering

Berger–Shor Algorithm

Theorem (stated without proof)

There exists at least one source and one sink in a DAG.

- This is a greedy algorithm which orders the vertices with sources first and sinks last, so that the direction of edges is enforced. This is exactly why we *append* vertices to S_l, but *prepend* vertices to S_r.
- Running Time: $O(|V| + |E|)$
- Can be modified to make sure that FAS is minimal by adding edges from strongly connected components to S_l and removing them from S_r. This is useful because edges crossing between two strongly connected components cannot be part of any cycle.
- This adds a running time of $O(|V||E|)$

Heuristics based on Vertex–Ordering

Greedy Cycle Removal

- Edges incident to either a sink or a source in a digraph cannot be part of a cycle
- Such vertices can be deleted recursively, which leads to a linear time algorithm.
- This sparser graph can be passed to the above approaches.

Heuristics based on Cycle–Breaking

Greedy_Edge_By_Edge

Input: Digraph $G := (V, E)$
Output: Optimised digraph with only those vertices which can potentially be part of a cycle

$S, T \leftarrow \emptyset$

for $e \in E$ **do**
    if $S \cup \{e\}$ is acyclic **then**
        append e **to** S
    else
        append e **to** T
    end if
end for

Heuristics based on Cycle–Breaking

Greedy Algorithm for Edge–by–Edge Cycle–Breaking

Theorem

1. *S is acyclic.*
2. *T is acyclic.*
3. *Both S and T are FASs and the smaller has at most $\frac{|E|}{2}$ edges.*
4. *T is a minimal FAS.*

Heuristics based on Cycle–Breaking

Proof of correctness.

1. S is acyclic by construction.
2. Since S is a DAG (from (1)), we can order the vertices at the endpoints of the edges in S by topological sort, say from left to right, based on their finishing/discovery times. Now, $\forall (u, v) \in T$, (u, v) would have closed a cycle in S (hence it was placed in T). This means that there was already a path from v to u using the edges in S, so v appears before u in the topological sort. Since every edge in T points in the opposite direction relative to S, T is acyclic.

$\square$

Heuristics based on Cycle–Breaking

Proof of correctness.

3. From (2), it is easy to see why S and T partition all the edges of G. Since S is a DAG, removing T from G leaves us S, and hence T is an FAS. From (2), T is also a DAG, so the converse holds.
   Since S and T partition E, $S \cup T = E \ \wedge \ S \cap T = \emptyset$
   $\Rightarrow |S| + |T| = |E| \wedge (|S| \leq |T| \vee |T| \leq |S|) \Rightarrow \min\{|S|, |T|\} \leq \frac{|E|}{2}$
4. By construction, we add an edge to T if and only if it closes a cycle in S. Because of the greedy approach, every edge in T can close a cycle in S. Hence, T has the least number of such edges.

$\square$

Layer Assignment

Definition

For a DAG (V, E), let $\mathcal{L} := \{L_0 \dots L_n\} : n \in \mathbb{N}$ be a partition of the vertex set E. If $(u, v) \in E : u \in L_j \wedge v \in L_i \Rightarrow i > j$, then $\mathcal{L}$ is called a ***layering*** of G and the subsets $L_0 \dots L_n$ are its ***layers***.

- A DAG attached with layers is called a *layered* DAG.
- Partitioning the vertex set to convert a DAG to its layered form is the *layer assignment* problem.

Layer Assignment

Definition

Let $G := (V, E, \lambda)$ be a DAG with a mapping $\lambda : V \rightarrow \{1 \ldots n\} : n \in \mathbb{N}$ such that:

1. $\{V_1 \ldots V_n\}$ is a partition of V
2. $\forall 1 \leq i \leq n, V_i = \lambda^{-1}(i)$
3. $\forall (u, v) \in E, \lambda(v) = \lambda(u) + 1$

Then, G is called a ***level graph***.

Definition

If $G := (V, E, \lambda)$ is a level graph such that $\forall 1 \leq i \leq n, v \in V_i, \exists (u, v) \in E : u \in V_{i-1}$, then G is called a ***hierarchy***.

Layer Assignment

Definition

Define a function $\ell : V \times 2^V \to \mathbb{N}$ such that $\ell(u, \mathcal{L})$ outputs the layer number containing the vertex u, i.e., $\ell(u, \mathcal{L}) = i \iff u \in L_i$. Then, $\ell(u, \mathcal{L})$ is called the ***rank*** of vertex u.

Definition

Define a function
$s : E \times 2^V \to \mathbb{N} : \forall e := (u, v) \in E, s(e, \mathcal{L}) = \ell(u, \mathcal{L}) - \ell(v, \mathcal{L})$. Then, $s(e, \mathcal{L})$ is called the ***span*** of edge e.
Trivially, the span of any edge is at least 1. If it is equal to 1, the edge is ***tight***, else it is ***long***.

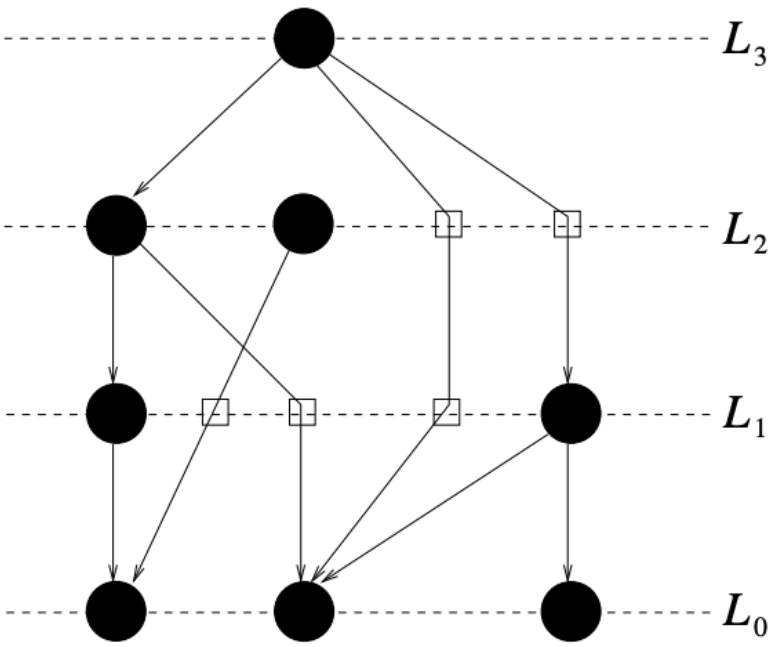
Definition

A layered DAG is ***proper*** if all of its edges are tight.

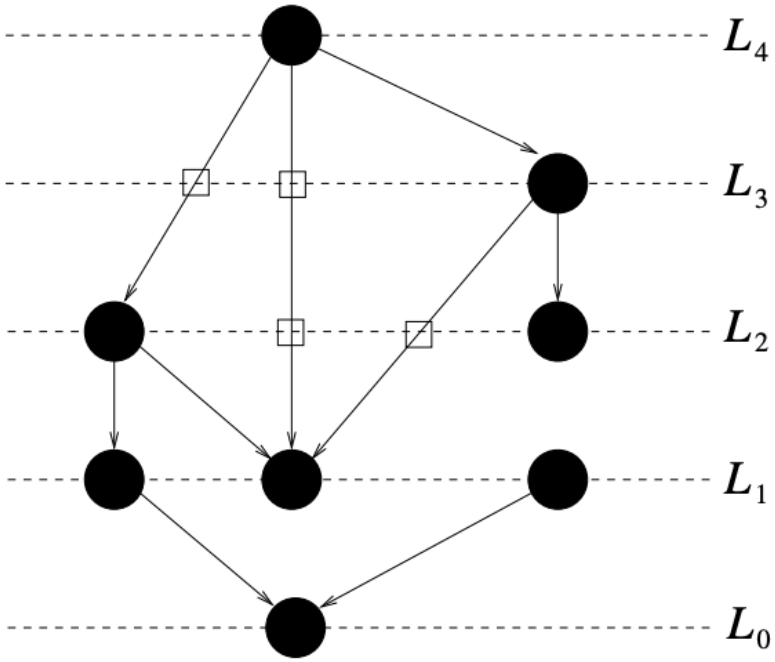
Layer Assignment

- The second step of the Sugiyama framework takes in a DAG (obtained from the previous step) and layers the graph with edges potentially criss–crossing.
- The next step, called *Vertex Ordering*, expects a proper layering and minimises the edge–crossings.
- It is trivial to see that every hierarchy is a proper layering.
- We can convert an improper layering to a proper one by adding dummy vertices, which sub–divide long edges.
- Let $e := (u, v)$ be a long edge in a layering $\mathcal{L}$ such that u and v have ranks j and i, respectively. We add dummy vertices $d_e^{i+1}, d_e^{i+2}, \ldots, d_e^{j-1}$ to the layers $L_{i+1}, L_{i+2}, \ldots, L_{j-1}$ respectively, such that the edge e is replaced by the path $\langle u, d_e^{j-1}, \ldots, d_e^{i+1}, v\rangle$.
- Define $\mathcal{D}(G, \mathcal{L})$ as the set of all dummy vertices introduced to a DAG G with a layering $\mathcal{L}$. Then, $|\mathcal{D}(G, \mathcal{L})| = \sum_{e \in E}(s(e, \mathcal{L}) - 1) = \sum_{e \in E} s(e, \mathcal{L}) - |E|$.

Layer Assignment



(a) A layered DAG with 4 layers and 5 dummy vertices.



(b) A layered DAG with 5 layers and 4 dummy vertices.

Unconstrained Layer Assignment

- This is an easy problem with existing solutions.
- Depends on modifying classical algorithms used for BFS, DFS and minimum spanning trees.
- Algorithms to be studied:
  - Modified Kahn's algorithm for topological sort
  - Modified BFS longest path algorithm
  - Modified DFS
  - Feasible Spanning Tree/Network Simplex

Constrained Layer Assignment

- In practice, certain constraints are imposed. For example, one such condition is minimising the number of dummy vertices, i.e, $|\mathcal{D}(G, \mathcal{L})|$. In further stages of the Sugiyama framework, too many dummy vertices may produce undesirable criss–crossing of edges.
- Algorithms to be studied:
  - Modified longest path algorithm
  - Coffman–Graham algorithm
  - Minimum width layering algorithm
  - Layering by promotion of vertices
  - Modified network simplex algorithm
  - Healy–Nikolov's branch–and–cut algorithm
  - Layering long edges by edge–splitting
- An optional step in the framework to reduce the edge density between adjacent layers, edge crossings and dummy vertices is called *Edge Concentration*.

Sources & References

1. AT&T Graphviz Homepage
2. Graphviz Gitlab Repository
3. Handbook of Graph Drawing & Visualisation – Department of Computer Science, Brown University
4. Layered Drawing of Graphs – Department of Computer Science, University of Crete
5. Graph Layering by Promotion of Nodes by Nikolov & Tarassov – Elsevier